

RealTwin-RL: Bridging Digital Twin Reinforcement Learning to Real-World VLA

Anonymous Authors

Abstract—Reinforcement learning (RL) post-training is important for improving the robustness of vision-language-action (VLA) policies. However, performing RL directly on real robots is often slow and expensive. To alleviate this, we explore RL post-training in digital twin environments, where the similarity to the real world and the ability to run many environments in parallel enable efficient large-scale rollouts. Simply scaling up rollouts in digital twins, however, does not necessarily produce effective RL supervision: the feedback can be both sparse in terms of task progress and misleading due to simulation-specific interactions. Therefore, in this paper, we propose RealTwin-RL, which extracts both progress-aware and reality-grounded learning signals from digital-twin rollouts, enabling efficient RL post-training toward behaviors that remain reliable in the real world. Specifically, we first introduce progress-aware milestone supervision, which learns task-progress estimation from semantic milestones and converts the predicted progress into dense potential-based rewards. These dense rewards improve policy performance under the same rollout budget. Furthermore, we propose a milestone-conditioned reality discriminator to evaluate whether observed progress is supported by interactions that remain feasible in the real world, preventing the policy from obtaining reward through digital-twin artifacts. Together, these components encourage RL to optimize for realizable task progress, rather than merely high reward in simulation. Experiments on five real-world tasks across two embodiments validate the effectiveness of the proposed method.

I. INTRODUCTION

Reinforcement learning (RL) post-training has become an important approach for improving the robustness and adaptability of vision-language-action (VLA) policies beyond supervised fine-tuning (SFT) by enabling optimization through environment interaction and task-level feedback [1], [2], [3], [4], [5]. However, scaling RL post-training on real robots remains challenging due to the high cost of repeated physical hardware interaction and the inherently slow execution cycles required to collect real-world rollouts [6], [7], [5], [8]. These limitations severely restrict the amount of experience available for policy optimization. To overcome these constraints, inspired by RialTo [7] and TwinRL [9], we investigate digital twin environments as a scalable platform for RL post-training of VLA policies, leveraging their efficient parallel rollouts, fast resets, and controllable interactions. This raises a fundamental question: how can we turn scalable digital-twin rollouts into effective RL supervision that improves policy learning while remaining transferable to the real world?

We observe that scalable rollout generation alone is not sufficient. As illustrated in Fig. 1, once interaction becomes abundant, the bottleneck shifts from collecting experience to extracting learning signals that are both efficient for policy optimization and aligned with behaviors that remain effective

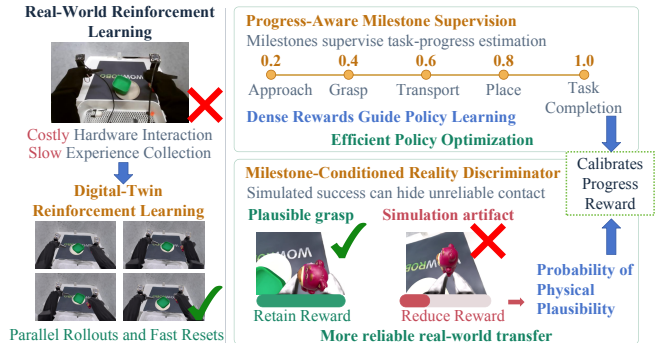


Fig. 1: **RealTwin-RL** addresses the limited rollout throughput of real-world RL by introducing parallel digital twins for scalable VLA policy rollouts (left). To ensure that the learned policy transfers reliably to the real world, RealTwin-RL further introduces semantic milestones to supervise a progress head for dense progress-based rewards (top right), together with a milestone-conditioned reality discriminator that calibrates rewards for contact-sensitive interactions (bottom right), promoting physically realizable task progress while reducing credit for simulator-specific behaviors.

on the real robot. This bottleneck has two aspects. **First**, sparse temporal feedback makes credit assignment difficult [10]. When rewards are observed only after long action sequences, the contribution of individual actions must be inferred through long-horizon value and advantage estimation, weakening the learning signal and slowing policy learning. Thus, increasing rollout throughput alone does not necessarily translate into proportional gains in sample efficiency. **Second**, simulator-derived rewards may not faithfully reflect physically realizable progress in the real world [11], [12], [13]. Residual mismatches in contact dynamics, friction, and object motion can cause the simulator to reward interactions that would fail on the real robot. Because RL directly optimizes these rewards, the policy may exploit such simulation-specific behaviors, achieving high simulated return without corresponding real-world improvement [14]. We therefore argue that effective RL supervision in digital twins must be both *progress-aware*, to provide informative learning signals throughout task execution, and *reality-calibrated*, to ensure that rewarded progress remains physically meaningful in the real world.

To this end, we propose **RealTwin-RL**, a framework for VLA RL post-training that transforms digital-twin rollouts into dense learning signals by estimating task progress and

calibrating its reward according to the physical plausibility of the underlying interactions. To start with, in order to provide more informative supervision throughout a complete rollout, we introduce **progress-aware milestone supervision**. Each manipulation task is represented as an ordered sequence of semantic milestones corresponding to meaningful stages of task completion, such as approaching the target object, establishing a grasp, moving the object toward the goal, and placing it at the desired location. Using milestone completion automatically detected from privileged simulator states as supervision, we train a progress head on the shared VLA representation and convert its predictions into dense potential-based rewards that guide credit assignment throughout task execution. This improves the efficiency with which rollout experience is converted into policy updates. However, because milestone completion is detected within the digital twin, progress-only supervision may still reward grasping or placement events that are successful only under simulator-specific dynamics.

Therefore, to address this limitation, RealTwin-RL uses a **milestone-conditioned reality discriminator** to assess, from visual evidence, whether interactions associated with contact-sensitive milestones are reproducible on the physical robot. Importantly, the resulting confidence score selectively down-weights the corresponding progress reward rather than acting as an additional reward, thereby calibrating the reliability of detected progress without adding a separate reward term to the policy objective. We train this discriminator on milestone keyframes using offline pseudo-labels generated by a vision-language model (VLM) from milestone-specific queries and privileged 3D geometry, enabling efficient plausibility assessment from the shared VLA representation during RL post-training. Overall, the progress-aware mechanism determines what progress has occurred, whereas the reality discriminator determines how strongly that progress should be credited. Together, these two components retain the scalability of digital-twin rollouts while guiding policy optimization toward progress that is both informative for learning and reliable in the real world.

Empirically, we evaluate RealTwin-RL on five manipulation tasks across two robotic embodiments. Compared with SFT, RealTwin-RL achieves consistent improvements in both digital-twin and real-world evaluations, increasing average task success rate by 25 percentage points on physical robots. RealTwin-RL further outperforms standard proximal policy optimization (PPO) [15] across the evaluated real-world tasks, demonstrating that progress-aware, reality-grounded supervision effectively converts scalable digital-twin interaction into policy improvements that reliably transfer to physical robots.

Our main contributions are summarized as follows:

- We propose **RealTwin-RL**, a digital-twin RL post-training framework for VLA policies that addresses sparse feedback and simulator-specific reward bias via progress-aware and reality-calibrated supervision.
- We introduce **Progress-Aware Milestone Supervision**, which learns task progress from automatically detected

semantic milestones and provides dense potential-based rewards for efficient RL optimization.

- We develop a **Milestone-Conditioned Reality Discriminator**, a lightweight head that assesses the physical plausibility of contact-sensitive milestone interactions and calibrates their progress rewards accordingly.
- Experiments across five manipulation tasks and two robotic embodiments show that RealTwin-RL outperforms SFT and standard PPO in real-world success, with ablations validating the benefits of both components.

II. RELATED WORK

A. Vision-Language-Action Models and RL Post-Training

Vision-language-action (VLA) models learn generalist policies that map visual observations and language instructions to robot actions [16], [17]. Reinforcement learning improves pretrained VLAs through environment interaction and task feedback [1], [2]. SimpleVLA-RL and RLux-VLA [3], [4] support scalable VLA reinforcement learning through parallel rollout collection and efficient optimization. Complementary post-training approaches combine offline and online fine-tuning in ConRFT [18], distill residual-RL recovery experience in PLD [19], and learn from autonomous real-world experience and corrections in $\pi_{0.6}^*$ [5].

Reward supervision determines how these interactions inform policy improvement. To address sparse feedback, VLA-RL [20] trains a robotic process reward model using automatically segmented successful trajectories and pseudo reward labels. Other sources of dense feedback include temporal rankings of passive videos in Rank2Reward [21], frozen VLM reasoning in GVL [22], and language-conditioned progress learning from demonstrations in ReWiND [23]. Stage- and progress-aware rewards are particularly relevant to long-horizon credit assignment. SARM [24] learns stage-aware progress from temporally annotated subtasks, while SRPO [25] assigns trajectory-level progress rewards by comparing rollouts with successful references in a world-model latent space. RealTwin-RL automatically detects completion of predefined semantic milestones from privileged digital-twin states and uses these targets to train a progress head on the shared VLA feature.

B. Digital Twins and Real-to-Sim-to-Real Robot Learning

Digital twins provide interactive virtual counterparts of real robotic workspaces, enabling robot learning to leverage scalable simulation while preserving scene-specific correspondence to the physical environment [7], [9], [26]. A key prerequisite is therefore accurate real-to-sim reconstruction and alignment. SplatSim and Re³Sim [27], [28] integrate Gaussian-based scene reconstruction with simulation, while Physically Embodied Gaussian Splatting [29] augments Gaussian representations with particle-based dynamics. TwinAligner [26] further reduces discrepancies between virtual and physical environments by aligning visual and physical properties through real-world interactions.

Beyond reconstructing a particular workspace, real-to-sim pipelines can also diversify reconstructed scenes and

trajectories to expand the data available for policy learning. ACDC [30] creates digital cousins for cross-scene variation; RoboSplat [31] augments demonstrations through Gaussian-scene and trajectory editing; and Real2Render2Real [32] renders training demonstrations from reconstructed assets and tracked motions. Real2Edit2Real [33] combines metric-scale reconstruction, spatial trajectory editing, and depth-controlled video generation. Together, these approaches extend reconstructed environments beyond faithful scene replication, increasing the visual and behavioral diversity available for downstream policy learning.

Building on such real-to-sim environments, digital twins can serve not only as sources of demonstrations but also as interactive platforms for policy optimization. RialTo [7] robustifies imitation policies through RL in scene-specific twins, and TwinRL [9] uses parallel twin-based VLA reinforcement learning to expand exploration and guide subsequent real-world training. RealTwin-RL focuses specifically on the reliability of RL supervision within digital-twin VLA post-training: it uses interactive digital twins to generate scalable RL experience, while combining semantic progress estimation with milestone-conditioned assessment of contact plausibility to ensure that policy optimization credits task progress that is likely to remain physically realizable on the real robot.

III. METHOD

A. RealTwin-RL Overview

Preliminaries. RealTwin-RL follows a supervised-to-reinforcement post-training pipeline for vision-language-action (VLA) policies. We first obtain an initial task-conditioned policy through supervised fine-tuning (SFT). The VLA policy consists of a vision-language backbone that encodes robot observations and language instructions into a shared multimodal representation, together with an action expert that predicts robot actions conditioned on this representation. Given a demonstration dataset

$$\mathcal{D} = \left\{ \left(\mathbf{o}_t^{(i)}, \mathbf{a}_t^{(i)}, l^{(i)} \right) \right\}, \quad (1)$$

where $\mathbf{o}_t^{(i)}$ denotes the robot observation, $\mathbf{a}_t^{(i)}$ the expert action, and $l^{(i)}$ the language instruction, we train the policy by supervised fine-tuning to obtain

$$\pi_{\theta_0} = \text{SFT}(\mathcal{D}). \quad (2)$$

Starting from the SFT-initialized policy π_{θ_0} , we further perform reinforcement learning post-training with Proximal Policy Optimization (PPO) [15] through interactions in task-specific digital twin environments, with randomized initial object poses to improve robustness to state variations.

Digital Twin Construction and Rollout. Following recent real-to-sim robotic systems [7], [27], [28], [26], we reconstruct task-specific digital twins from the corresponding real-world manipulation scenes to support scalable RL post-training. The robot and task objects are instantiated in the Genesis physics engine [34] for contact and dynamics, while their visual appearance is independently represented using

per-asset 3D Gaussian Splatting (3DGS) reconstructions [35] obtained from real-world captures. Within these digital-twin environments, we perform policy rollouts for RL post-training. At each rollout step t , the policy predicts an action chunk of horizon H conditioned on the current observation and language instruction,

$$\mathbf{a}_{t:t+H-1} \sim \pi_{\theta}(\cdot \mid \mathbf{o}_t, l). \quad (3)$$

The predicted action chunk is then executed in the digital twin, where Genesis updates the robot and object poses according to the simulated dynamics. The corresponding 3DGS assets are transformed based on these updated poses and rendered to produce the next visual observation. Therefore, parallel digital-twin environments enable scalable rollout collection with efficient resets and task-specific interactions. To improve robustness against the distribution gap between real and digital-twin rollouts, we apply visual augmentation during SFT and randomize initial configurations during RL rollouts. Despite these measures, contact dynamics remain imperfectly matched, causing simulator-specific interactions that produce misleading task progress.

Method Overview. While digital-twin environments enable scalable collection of interaction experience, translating this scalability into sample-efficient RL post-training requires reward supervision that addresses two key challenges: sparse temporal feedback and simulator-specific reward bias. To this end, RealTwin-RL augments the shared vision-language representation of the SFT-initialized VLA policy with two lightweight auxiliary heads built upon task-specific semantic milestones, as illustrated in Fig. 2. The *Progress-Aware Milestone Supervision* module (Sec. III-B) introduces a progress head to estimate task advancement throughout execution, providing dense intermediate supervision that alleviates the multi-step credit-assignment difficulty induced by sparse task-level rewards. Complementarily, the *Milestone-Conditioned Reality Discriminator* (Sec. III-C) evaluates the physical realizability of milestone interactions and calibrates the corresponding reward credit, reducing the influence of progress arising from simulator-specific behaviors. Together, these two components transform digital-twin rollouts into *progress-aware* and *reality-calibrated* reward supervision. The calibrated rewards are then used for advantage estimation and PPO optimization, as detailed in Sec. III-D.

B. Progress-Aware Milestone Supervision

Task-level success signals provide only sparse temporal feedback, making it difficult to determine which intermediate actions contribute to eventual task completion during execution. To alleviate this credit-assignment challenge, we introduce a lightweight progress head that estimates task advancement from the shared vision-language representation, providing dense intermediate supervision throughout execution. Importantly, our goal is not to directly expose simulator-derived dense rewards to the policy, but to enable the policy to infer its current task-completion stage from its own visual-language observations, thereby developing an explicit awareness of task progress that can better guide

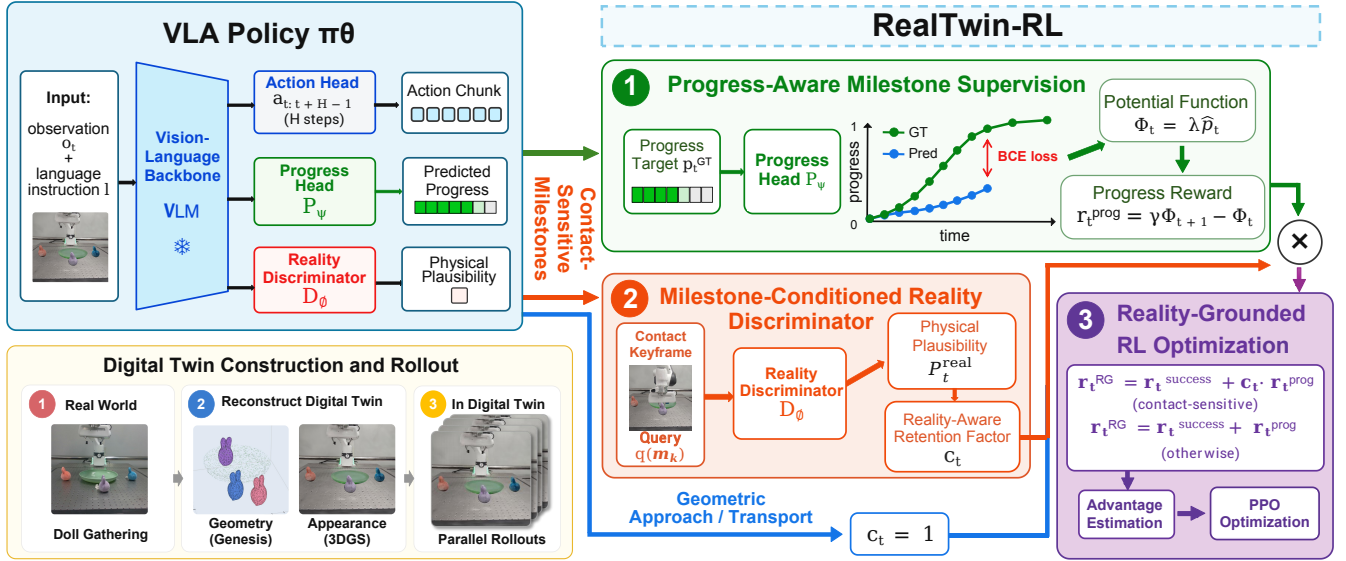


Fig. 2: Overview of RealTwin-RL. Task-specific digital twins combine Genesis physics and 3DGS rendering for scalable parallel rollouts. Progress-Aware Milestone Supervision provides dense progress rewards, while the Milestone-Conditioned Reality Discriminator calibrates contact-sensitive rewards. The resulting rewards are used for PPO optimization.

subsequent action prediction.

Milestone Detection and Progress Targets. We exploit privileged robot and object states in the digital twin to automatically identify semantic milestone completion and use these milestones as supervision for learning progress estimation. Specifically, for each task $\mathcal{T}$, we define a short ordered sequence of semantic milestones,

$$\mathcal{M}_{\mathcal{T}} = (m_1, m_2, \dots, m_{K_{\mathcal{T}}}), \quad (4)$$

where $K_{\mathcal{T}}$ denotes the number of milestones for task $\mathcal{T}$. Each milestone m_k is assigned a predefined reward weight w_k , reflecting its contribution to task progress. These milestones correspond to semantically meaningful stages of task execution, such as grasping an object or moving it toward the target. Each milestone is associated with a simple state-based completion predicate defined once before RL post-training and reused across all subsequent rollouts. During digital-twin rollouts, we automatically detect milestone completion using privileged robot and environment states available from the simulator, such as robot velocity, gripper state, relative object pose, and object configuration. This provides scalable progress supervision without laborious frame-level annotation. When a milestone is first completed, the corresponding visual observation is recorded as a milestone keyframe. The milestone completion level is used to supervise task-progress estimation, while the associated keyframe is retained for subsequent reality evaluation (Sec. III-C). Let $k_t \in \{0, \dots, K_{\mathcal{T}}\}$ denote the index of the latest milestone completed by time t . We define the normalized progress target as

$$p_t^{\text{GT}} = \frac{\sum_{j=1}^{k_t} w_j}{\sum_{j=1}^{K_{\mathcal{T}}} w_j}, \quad (5)$$

where $p_t^{\text{GT}} \in [0, 1]$ represents the fraction of cumulative

milestone reward achieved by time t . This provides explicit supervision for learning task progress from the policy’s own visual-language observations.

Progress Estimation. To learn this progress representation, we attach a lightweight progress head P_{ψ} to the shared vision-language backbone. Given the representation h_t extracted from the backbone, the progress head predicts

$$\hat{p}_t = P_{\psi}(h_t), \quad (6)$$

where $\hat{p}_t \in [0, 1]$ is obtained through a sigmoid output. We train the progress head using the auxiliary objective

$$\mathcal{L}_{\text{PH}} = \text{BCE}(\hat{p}_t, p_t^{\text{GT}}). \quad (7)$$

Although mean squared error (MSE) is commonly used for scalar regression, combining it with a sigmoid output can lead to attenuated gradients near the boundaries. This issue is particularly relevant to milestone-based supervision, which naturally contains boundary targets, e.g., $p_t^{\text{GT}} = 0$ at task initialization and $p_t^{\text{GT}} = 1$ upon completion. We therefore adopt binary cross-entropy (BCE) over the normalized progress targets for more stable optimization.

Progress-Head Training. After SFT, we pretrain the progress head using milestone-derived targets while freezing the VLA backbone and optimizing only the head with $\mathcal{L}_{\text{PH}}$. During RL, the progress head is jointly updated with the policy using milestone-derived targets and the same auxiliary objective $\mathcal{L}_{\text{PH}}$. This enables the progress head to adapt to the evolving rollout distribution as the policy improves.

Progress-Based Reward Shaping. The learned progress prediction is then used to construct a dense shaping reward. Specifically, we define a potential function

$$\Phi_t = \lambda \hat{p}_t, \quad (8)$$

where λ is a task-dependent scaling factor that matches the

range of the potential function to the total milestone reward scale. Following potential-based reward shaping [36], the progress reward is computed as

$$r_t^{\text{prog}} = \gamma \Phi_{t+1} - \Phi_t, \quad (9)$$

where γ is 1.0. Since the policy predicts action chunks, we compute r_t^{prog} at the chunk level and incorporate it into the reward used for PPO optimization.

While this progress-aware reward provides dense feedback, task progress may still arise from simulator-specific contact interactions and thus may not be physically realizable. We address this issue by calibrating the progress-based reward according to the estimated physical plausibility of the underlying interaction, as described in the next section.

C. Milestone-Conditioned Reality Discriminator

To assess the physical realizability of milestone-associated interactions, we introduce a milestone-conditioned reality discriminator. However, not all milestones are equally affected by the sim-to-real gap. Non-contact milestones (e.g., approach and transport) do not establish new object contacts, while contact-sensitive milestones (e.g., grasp and place) rely on simulated contact dynamics and are more vulnerable to simulator-specific artifacts. Therefore, we apply reward calibration selectively to contact-sensitive milestones rather than all milestones. This selective design avoids modifying reliable geometric progress signals while focusing discriminator evaluation on interactions prone to reality mismatch.

Training Data Collection. We train the reality discriminator offline before RL post-training. To construct the training set, we roll out the SFT-initialized policy in the digital twin and collect milestone keyframes whenever a contact-sensitive milestone m_k is triggered.

Pseudo-Label Construction. To obtain supervision for each collected interaction, we exploit the privileged 3D geometry available in the digital twin. Specifically, we extract milestone-relevant geometry from the simulator state and construct a milestone-specific query $q(m_k)$ for an external VLM evaluator (Doubao-Seed-2.0-Pro). The query combines a semantic interaction description, e.g., “Determine whether the gripper is grasping the blue doll,” with milestone-specific spatial criteria and corresponding world-frame surface point clouds of relevant entities (e.g., the target object and gripper fingers for grasping milestones). The point clouds are downsampled to a fixed vertex budget (e.g., 100 points for the target object and 48 points per gripper finger) to reduce input complexity while preserving relevant spatial relations. Given this query, the external VLM predicts whether the interaction is physically realizable and provides an uncertainty estimate. To assess the reliability of this automatically generated supervision, we randomly sample 200 milestone interactions across all tasks and compare the VLM judgments with human annotations. Most disagreements arise when the evaluator judges an interaction as unsuccessful even though human annotators consider it acceptable. We find that such conservative judgments do not negatively affect reward shaping, as they mainly reduce reward credit for borderline

interactions rather than incorrectly rewarding physically implausible ones.

Reality Discriminator Training. Given the resulting pseudo-labeled milestone data, we train a lightweight reality discriminator head D_ϕ attached to the shared VLA representation. For a triggered milestone m_k , let $\mathbf{h}_t^{m_k}$ denote the corresponding representation extracted by the VLA backbone. The discriminator predicts

$$p_t^{\text{real}} = \sigma(D_\phi(\mathbf{h}_t^{m_k})), \quad (10)$$

where $p_t^{\text{real}} \in [0, 1]$ denotes the predicted probability that the milestone-associated interaction is physically realizable.

The discriminator is trained using a class-weighted binary cross-entropy objective,

$$\mathcal{L}_{\text{RD}} = -w_{y_t} [y_t \log p_t^{\text{real}} + (1 - y_t) \log(1 - p_t^{\text{real}})], \quad (11)$$

where $y_t \in \{0, 1\}$ denotes the pseudo-label generated by the external VLM evaluator and w_{y_t} accounts for class imbalance. Importantly, the external VLM and privileged geometry are used only for offline label construction.

Reality-Aware Reward Calibration. During RL post-training, when a contact-sensitive milestone m_k is triggered, the frozen discriminator predicts its physical plausibility p_t^{real} . We use this probability to continuously calibrate the corresponding progress reward via

$$c_t = m_{\min} + (1 - m_{\min})p_t^{\text{real}}, \quad (12)$$

where $m_{\min} > 0$ sets the minimum retained reward fraction. Thus, physically plausible interactions retain nearly the full progress reward, while less plausible ones receive reduced credit. The non-zero lower bound preserves learning signals under imperfect predictions and avoids the abrupt changes introduced by hard accept-or-reject thresholding. The resulting reality-aware retention factor c_t is used to calibrate the progress-based reward and is incorporated into the final RL objective described in the following section.

D. Reality-Grounded RL Optimization

The progress estimator (Sec. III-B) and reality discriminator (Sec. III-C) provide complementary signals: the former measures how much task progress is achieved, while the latter evaluates whether such progress is physically credible. The terminal task-completion reward r_t^{success} is retained as the sparse success signal, while the discriminator output c_t calibrates progress rewards for contact-sensitive interactions. We therefore define the reality-grounded reward as

$$r_t^{\text{RG}} = r_t^{\text{success}} + \begin{cases} c_t r_t^{\text{prog}}, & \text{if contact-sensitive,} \\ r_t^{\text{prog}}, & \text{otherwise.} \end{cases} \quad (13)$$

This formulation can be viewed as weighting task progress by its real-world credibility while preserving the terminal success reward. Progress achieved through physically plausible interactions retains nearly full credit, whereas progress arising from simulator-specific contact artifacts is down-weighted. The resulting rewards are subsequently used for advantage estimation and PPO-based policy optimization.

IV. EXPERIMENTS

We organize the experiments around three questions:

- (1) Does RealTwin-RL improve real-world task performance over SFT and standard PPO?
- (2) How do progress-aware supervision and reality-aware reward calibration contribute to the real-world improvement?
- (3) Are the gains from RealTwin-RL over the SFT initialization consistent across digital-twin and real-world evaluations?

A. Experimental Setup

We use $\pi_{0.5}$ [37] as the base VLA model. The initial SFT policy for each task is obtained by fine-tuning the base VLA model on 30–50 demonstration trajectories. We evaluate RealTwin-RL on five manipulation tasks using a dual-arm XLeRobot and a Franka robotic arm, as illustrated in Fig. 3. The XLeRobot performs three tasks. In *Bowl Stacking*, it places a middle bowl into a base bowl and then a top bowl into the middle bowl, forming a three-bowl stack. In *Bimanual Pouring*, it holds a bowl with the left arm and a cup with the right arm, then pours beads from the cup into the bowl. In *Bowl and Bear Placement*, it first places a bowl on a plate and then places a bear toy into the bowl. The Franka performs the other two tasks. In *Doll Gathering*, it places three dolls on a plate. In *Drawer Stowing*, it opens a partially open drawer, places a bitter melon inside, and closes the drawer.

For each task, all methods are evaluated on the physical robot using a shared set of test conditions, with 20 trials per method. Table I and Table II report real-world success rates for the main comparison and ablation study, respectively, together with the average across the five tasks. For the cross-domain comparison in Table III, we evaluate the same SFT policy and the same final RealTwin-RL checkpoint in the digital twin and on the physical robot.

B. Main Comparison on Physical Robots

The main experiment evaluates the effectiveness of digital-twin RL post-training on physical robots. We compare the SFT policy, standard PPO initialized from the same SFT policy and trained in the digital twin, and RealTwin-RL (ours). Within each task, all RL variants share the same SFT initialization and rollout budget. We evaluate checkpoints from the same training iteration and deploy the corresponding policies without further optimization.

Table I shows that RealTwin-RL outperforms both SFT and standard PPO on all five tasks, with an average improvement of 23 percentage points over PPO. In contrast, standard PPO provides only limited and task-dependent improvements over SFT under the evaluated rollout budget. Because the RL methods share their initialization and online interaction budget, this contrast highlights the importance of reward supervision in converting digital-twin experience into real-world policy improvement. Simply collecting more online interaction is not sufficient to achieve the gains obtained by the proposed method.

TABLE I: **Comparisons with baselines on physical robots.** Task success rates (%) are evaluated over 20 trials per task.

Task	SFT	PPO	Ours
Bowl Stacking	55	55	85
Bimanual Pouring	65	70	90
Bowl and Bear Placement	45	50	60
Doll Gathering	50	45	75
Drawer Stowing	35	40	65
Average	50	52	75

TABLE II: **Ablation study of progress supervision and reality-aware calibration.** All configurations are trained with PPO in the digital twin and evaluated on physical robots.

Configuration	Ex0	Ex1	Ex2	Ex3 (Ours)
Progress head (PH)	–	✓	–	✓
Direct milestone reward	–	–	✓	–
Reality-aware calibration	–	–	–	✓
Task	Success rate (%) ↑			
Bowl Stacking	55	75	65	85
Bimanual Pouring	70	70	80	90
Bowl and Bear Placement	50	50	45	60
Doll Gathering	45	60	65	75
Drawer Stowing	40	50	60	65
Average	52	61	63	75

The five tasks require more than isolated successful actions: stacking and drawer stowing involve dependent manipulation stages, doll gathering repeats a sequence of object transfers, and pouring requires coordinated object handling with both arms. In these settings, an intermediate action matters not only because it advances the current stage, but also because it establishes the state from which subsequent operations must succeed. RealTwin-RL addresses these two requirements by estimating task advancement and calibrating the reward assigned to contact-sensitive progress. The progress signal provides intermediate feedback, while reality-aware calibration reduces positive reward credit for interactions predicted to be physically implausible.

C. Ablation Study on Physical Robots

We examine the contribution of progress supervision and reality-aware calibration through four configurations in Table II. All configurations use PPO for policy optimization in the digital twin. Within each task, they are initialized from the same SFT policy and trained with the same environment interaction budget. All policies are evaluated on physical robots.

Ex0: PPO with task-level rewards. The first configuration uses the task-level sparse reward without intermediate progress feedback or reality-aware calibration. This configuration establishes the reference for assessing how each following component improves RL post-training in the digital twin.

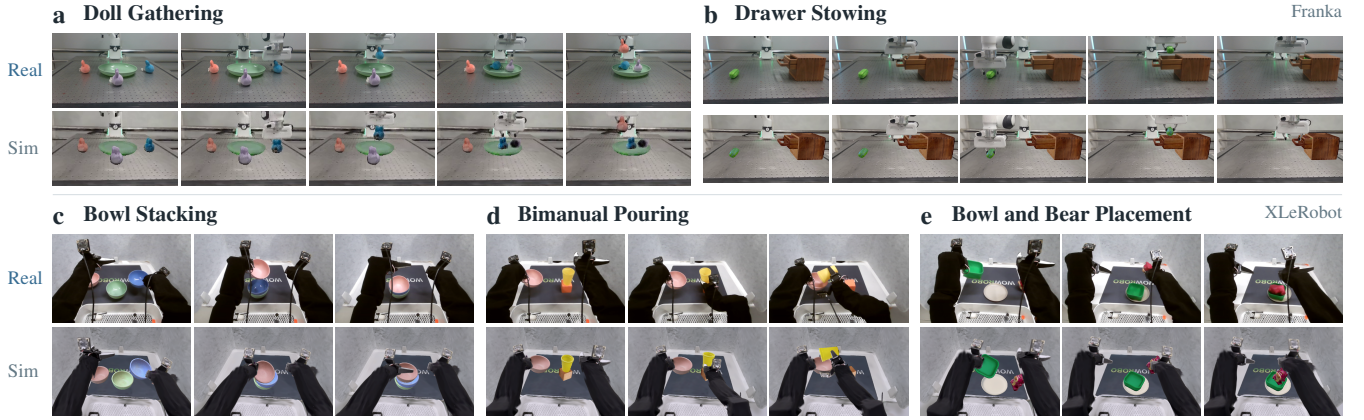


Fig. 3: Overview of the five manipulation tasks. The top row shows the Franka tasks, and the bottom row shows the XLeRobot tasks. Real and Sim denote real-world observations and digital-twin renderings, respectively.

Ex1: With learned progress prediction. Compared with Ex0, Ex1 adds the progress head (Sec. III-B) to provide shaping rewards, while reality-aware calibration is disabled. In most tasks, we observe an obvious success gain. These gains demonstrate the effectiveness of learned progress feedback for credit assignment across sequential manipulation stages. However, success rates remain unchanged for Bimanual Pouring (70%) and Bowl and Bear Placement (50%), suggesting that providing denser reward feedback alone is insufficient: the learned policy may exploit behaviors that are effective in the digital twin but do not transfer reliably to the real world.

Ex2: With direct milestone rewards. This configuration uses the same milestone reward magnitude as Ex1, with the only difference being the source of the reward. In Ex1, the reward is derived from the learned progress prediction, whereas here it is obtained heuristically from simulator-detected milestone completion. This comparison therefore isolates the effect of the learned progress head. Ex2 slightly outperforms Ex1 on several tasks, as its milestone rewards are computed from privileged simulator states and therefore provide cleaner supervision. However, such heuristic rewards rely on task-specific state access and manually defined predicates, making them difficult to scale to real-world RL. In contrast, the progress head infers task advancement directly from visual-language observations, providing a more scalable reward signal without privileged state information.

Ex3: With reality-aware calibration (Ours). Building on Ex1, Ex3 applies the milestone-conditioned reality discriminator to calibrate progress rewards at contact-sensitive interactions, as defined in Sec. III-D. Compared with Ex1, adding Reality-Aware Reward Calibration leads to clear improvements across all tasks, demonstrating that evaluating the physical plausibility of rewarded progress is important for preventing simulator-specific behaviors from being reinforced and for improving real-world policy performance.

D. Cross-Domain Improvement from SFT to RealTwin-RL

We examine whether the improvement from the SFT initialization to the final RealTwin-RL checkpoint is observed in

TABLE III: **Consistency of RL gains from simulation to the real world.** We compare the improvement from the SFT policy to the RealTwin-RL policy in both digital twin and real-world evaluation. Values are task success rates (%).

Task	Sim		Real	
	SFT	Ours	SFT	Ours
Bowl Stacking	25	40	55	85
Bimanual Pouring	55	65	65	90
Bowl and Bear Placement	40	45	45	60
Doll Gathering	55	65	50	75
Drawer Stowing	20	30	35	65
Average	39	49	50	75

both digital-twin and real-world evaluations. For a controlled comparison, we evaluate the same SFT policy and the same final RealTwin-RL checkpoint in both domains. As shown in Table III, RealTwin-RL improves over SFT on all five tasks in both domains. This consistency in the direction of improvement between simulation and reality supports the transferability of gains achieved through digital-twin RL.

V. CONCLUSION

We introduce RealTwin-RL, a digital-twin reinforcement learning framework that enables efficient VLA post-training through progress-aware and reality-grounded supervision. By extracting dense task-progress signals while calibrating reward credit for physically unreliable interactions, the framework makes scalable digital-twin experience more effective for real-world policy improvement. Experiments across five manipulation tasks and two robotic embodiments validate the effectiveness of the proposed approach. Our current implementation nevertheless inherits limitations from digital-twin construction, as accurately modeling highly deformable objects, fluids, and complex contact dynamics remains challenging. Extending the framework to richer digital-twin representations and broader classes of contact-rich manipulation is an important direction for future work.

REFERENCES

- [1] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu *et al.*, “Improving vision-language-action model with online reinforcement learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. IEEE, 2025, pp. 15665–15672, doi: 10.1109/ICRA55743.2025.11127299.
- [2] J. Liu, F. Gao, B. Wei, X. Chen, Q. Liao, Y. Wu *et al.*, “What can RL bring to VLA generalization? An empirical study,” in *Adv. Neural Inf. Process. Syst.*, vol. 38. Curran Associates, Inc., 2025, pp. 97121–97151, doi: 10.52202/085713-3248.
- [3] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang *et al.*, “SimpleVLA-RL: Scaling VLA training via reinforcement learning,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026, pp. 125846–125869.
- [4] H. Zang, M. Wei, S. Xu, Y. Wu, Z. Guo, Y. Wang *et al.*, “RLux-VLA: A unified and efficient framework for reinforcement learning of vision-language-action models,” in *Proc. Robot.: Sci. Syst. (RSS)*, Sydney, Australia, Jul. 2026, doi: 10.15607/RSS.2026.XXII.089.
- [5] A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. B. Connors *et al.*, “ $\pi_{0.6}^*$: a VLA that learns from experience,” in *Proc. Robot.: Sci. Syst. (RSS)*, Sydney, Australia, Jul. 2026, doi: 10.15607/RSS.2026.XXII.087.
- [6] G. Dulac-Arnold, N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Goyal *et al.*, “Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis,” *Mach. Learn.*, vol. 110, no. 9, pp. 2419–2468, 2021, doi: 10.1007/s10994-021-05961-4.
- [7] M. Torne Villasevil, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta *et al.*, “Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation,” in *Proc. Robot.: Sci. Syst. (RSS)*, Delft, Netherlands, Jul. 2024, doi: 10.15607/RSS.2024.XX.015.
- [8] A. K. Sharma, Y. Sun, N. Lu, Y. Zhang, J. Liu, and S. Yang, “World-gymnast: Training robots with reinforcement learning in a world model,” in *Proc. 2nd Workshop World Models, ICLR*, 2026.
- [9] Q. Xu, J. Liu, R. Zhou, S. Shi, N. Han, Z. Liu *et al.*, “TwinRL: Digital twin-driven reinforcement learning for real-world robotic manipulation,” in *Proc. 34th ACM Int. Conf. Multimedia*, 2026, to be published.
- [10] J. A. Arjona-Medina, M. Gillhofer, M. Widrich, T. Unterthiner, J. Brandstetter, and S. Hochreiter, “RUDDER: Return decomposition for delayed rewards,” in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 13566–13577.
- [11] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 3803–3810, doi: 10.1109/ICRA.2018.8460528.
- [12] P. Huang, X. Zhang, Z. Cao, S. Liu, M. Xu, W. Ding *et al.*, “What went wrong? Closing the sim-to-real gap via differentiable causal discovery,” in *Proc. 7th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 229. PMLR, 2023, pp. 734–760.
- [13] E. Aljalbout, J. Xing, A. Romero, I. Akinola, C. R. Garrett, E. Heiden *et al.*, “The reality gap in robotics: Challenges, solutions, and best practices,” *Annu. Rev. Control Robot. Auton. Syst.*, vol. 9, no. 1, pp. 403–432, 2026, doi: 10.1146/annurev-control-031924-100130.
- [14] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” 2016, arXiv:1606.06565, doi: 10.48550/arXiv.1606.06565.
- [15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [16] K. Kawaharazuka, J. Oh, J. Yamada, I. Posner, and Y. Zhu, “Vision-language-action models for robotics: A review towards real-world applications,” *IEEE Access*, vol. 13, pp. 162467–162504, 2025, doi: 10.1109/ACCESS.2025.3609980.
- [17] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 2165–2183.
- [18] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, “ConRFT: A reinforced fine-tuning method for VLA models via consistency policy,” in *Proc. Robot.: Sci. Syst. (RSS)*, Los Angeles, CA, USA, Jun. 2025, doi: 10.15607/RSS.2025.XXI.019.
- [19] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie *et al.*, “Self-improving vision-language-action models with data generation via residual RL,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.
- [20] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao *et al.*, “VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning,” 2025, arXiv:2505.18719, doi: 10.48550/arXiv.2505.18719.
- [21] D. Yang, D. Tjia, J. Berg, D. Damen, P. Agrawal, and A. Gupta, “Rank2Reward: Learning shaped reward functions from passive video,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. IEEE, 2024, pp. 2806–2813, doi: 10.1109/ICRA57147.2024.10610873.
- [22] Y. J. Ma, J. Hejna, C. Fu, D. Shah, J. Liang, Z. Xu *et al.*, “Vision language models are in-context value learners,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, pp. 33984–34009.
- [23] J. Zhang, Y. Luo, A. Anwar, S. A. Sontakke, J. J. Lim, J. Thomason *et al.*, “ReWiND: Language-guided rewards teach robot policies without new demonstrations,” in *Proc. 9th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 305. PMLR, 2025, pp. 460–488.
- [24] Q. Chen, J. Yu, M. Schwager, P. Abbeel, F. Shentu, and P. Wu, “SARM: Stage-aware reward modeling for long horizon robot manipulation,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026, pp. 17091–17116.
- [25] S. Fei, S. Wang, L. Ji, A. Li, S. Zhang, L. Liu *et al.*, “SRPO: Self-referential policy optimization for vision-language-action models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2026, pp. 6718–6728.
- [26] H. Fan, H. Dai, J. Zhang, J. Li, Q. Yan, Y. Zhao *et al.*, “TwinAligner: Visual-dynamic alignment empowers physics-aware Real2Sim2Real for robotic manipulation,” 2025, arXiv:2512.19390, doi: 10.48550/arXiv.2512.19390.
- [27] M. N. Qureshi, S. Garg, F. Yandun, D. Held, G. Kantor, and A. Silwal, “SplatSim: Zero-shot Sim2Real transfer of RGB manipulation policies using Gaussian splatting,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 6502–6509, doi: 10.1109/ICRA55743.2025.11128339.
- [28] X. Han, J. Yu, M. Liu, Y. Chen, X. Lyu, Y. Tian *et al.*, “Re³Sim: Generating high-fidelity simulation data via 3D-photorealistic real-to-sim for robotic manipulation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.
- [29] J. Abou-Chakra, K. Rana, F. Dayoub, and N. Suenderhauf, “Physically embodied Gaussian splatting: A visually learnt and physically grounded 3D representation for robotics,” in *Proc. 8th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 270. PMLR, 2025, pp. 513–530.
- [30] T. Dai, J. Wong, Y. Jiang, C. Wang, C. Gokmen, R. Zhang *et al.*, “Automated creation of digital cousins for robust policy learning,” in *Proc. 8th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 270. PMLR, 2025, pp. 4912–4943.
- [31] S. Yang, W. Yu, J. Zeng, J. Lv, K. Ren, C. Lu *et al.*, “Novel demonstration generation with Gaussian splatting enables robust one-shot manipulation,” in *Proc. Robot.: Sci. Syst. (RSS)*, Los Angeles, CA, USA, Jun. 2025, doi: 10.15607/RSS.2025.XXI.146.
- [32] J. Yu, L. Fu, H. Huang, K. El-Refai, R. A. Ambrus, R. Cheng *et al.*, “Real2Render2Real: Scaling robot data without dynamics simulation or robot hardware,” in *Proc. 9th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 305. PMLR, 2025, pp. 547–577.
- [33] Y. Zhao, H. Fan, D. Chen, S. Chen, L. Chen, X. Li *et al.*, “Real2Edit2Real: Generating robotic demonstrations via a 3D control interface,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2026, pp. 23106–23116.
- [34] Genesis Authors, “Genesis: A generative and universal physics engine for robotics and beyond,” GitHub, Dec. 2024, Accessed: Sep. 17, 2026. [Online]. Available: <https://github.com/Genesis-Embodied-AI/genesis-world>
- [35] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, Jul. 2023, Art. no. 139, doi: 10.1145/3592433.
- [36] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proc. 16th Int. Conf. Mach. Learn. (ICML)*. Morgan Kaufmann, 1999, pp. 278–287.
- [37] K. Black, N. Brown, J. Darpanian, K. Dhabalia, D. Driess, A. Esmail *et al.*, “ $\pi_{0.5}^*$: a vision-language-action model with open-world generalization,” in *Proc. 9th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 305. PMLR, 2025, pp. 17–40.